How to Improve the Current Mini SOC Lab

Action plan for upgrading the Wazuh SIEM/XDR portfolio project

Project: enterprise-soc-lab-wazuh-mitre-detection

1. Purpose of This Document

This document is focused only on how to improve the current Mini SOC lab. The current project already shows Wazuh SIEM/XDR deployment, Windows and Ubuntu endpoints, six attack simulations, MITRE ATT&CK mapping, screenshots, and incident reports. The next step is to make the lab look more enterprise-like, improve detection depth, and add stronger analyst evidence.

Current Lab Strength	Improvement Goal
Wazuh 4.7.5 SIEM/XDR is running	Make the lab more realistic with active response, deeper endpoint telemetry, and better evidence collection
Windows 10 endpoint with Sysmon	Tune Sysmon and add PowerShell logging for stronger Windows visibility
Ubuntu endpoint monitored by Wazuh agent	Add auditd for deeper Linux command, file, and privilege monitoring
Six attacks documented	Add raw alert details, JSON evidence, and response steps for each incident
GitHub README is built	Keep the repo clean, professional, and easy for recruiters to understand

2. Priority Roadmap

Use this order so the improvements build on top of each other instead of becoming messy.

Priority	Improvement	Impact	Difficulty
1	Clean GitHub evidence and remove sensitive screenshots	High — improves recruiter trust immediately	Easy
2	Add raw alert details and JSON evidence for each attack	High — makes the project look like real SOC triage	Easy

3	Enable Wazuh Active Response for brute-force blocking	High — shows response, not only detection	Medium
4	Improve Windows monitoring with PowerShell logging and tuned Sysmon	High — stronger Windows blue-team coverage	Medium
5	Add Linux auditd to Ubuntu endpoint	High — deeper Linux telemetry	Medium
6	Create and test custom Wazuh rules with custom rule IDs	High — shows detection engineering	Medium
7	Add a Kali attacker VM	Medium — makes the lab architecture more realistic	Easy/Medium
8	Add email or Slack alerting for high-severity alerts	Medium — shows SOC notification workflow	Medium
9	Build vulnerability remediation workflow	Medium — shows vulnerability management skills	Easy
10	Add final executive report and portfolio summary	High — helps recruiters understand the outcome quickly	Easy

3. Planned Improved Architecture

The improved version of the lab should keep the same VirtualBox host-only network, but add stronger monitoring and response capability. The Windows endpoint should continue using Sysmon, while the Ubuntu endpoint should be upgraded with auditd. A Kali attacker VM can also be added later for controlled attack simulation.

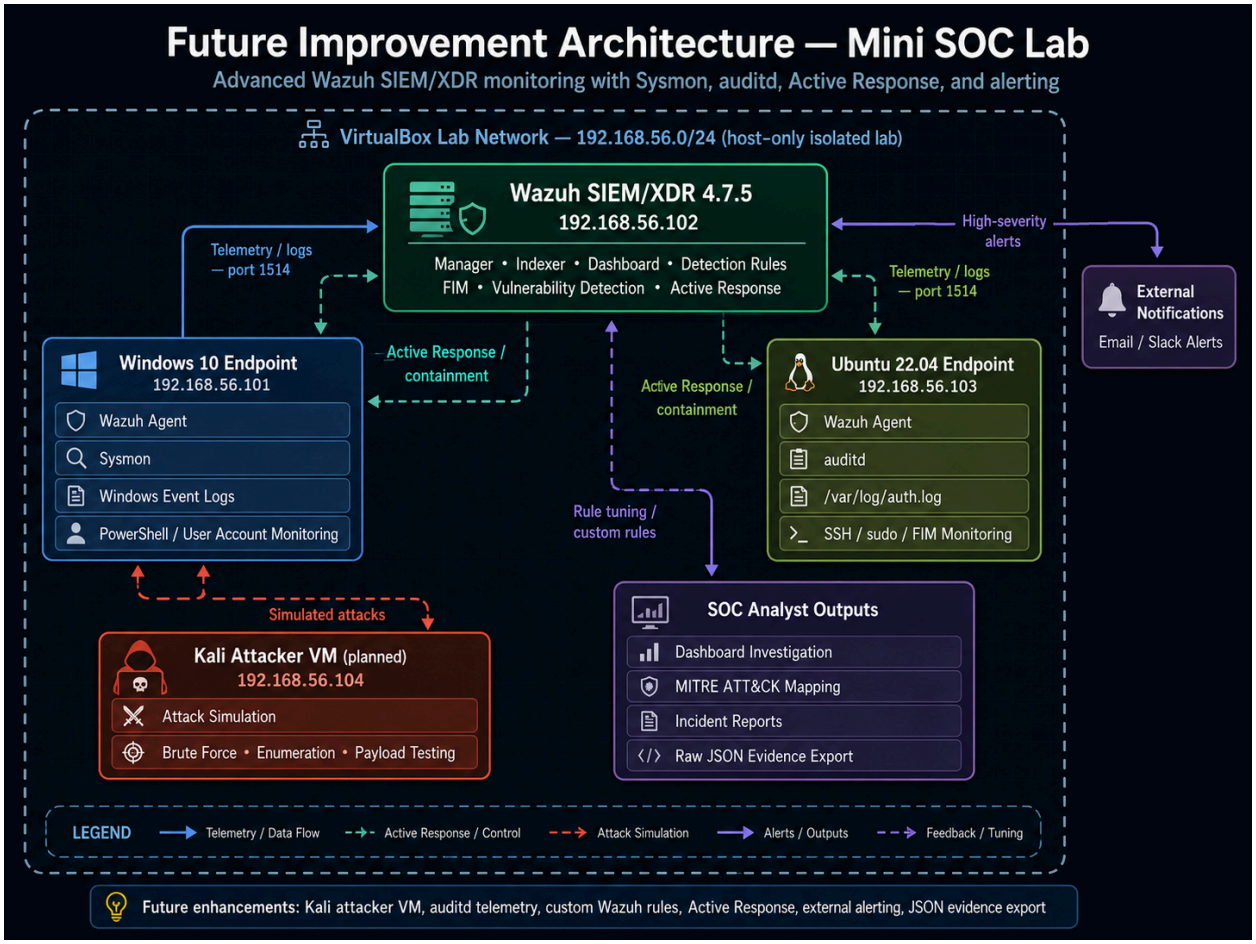

Figure 1: Planned future architecture with Windows Sysmon and Linux auditd monitoring.

Machine	Current Role	Improvement
Wazuh Server — 192.168.56.102	Manager, Indexer, Dashboard	Add Active Response, alerting, cleaner rule testing, and raw alert exports
Windows 10 — 192.168.56.101	Windows logs and Sysmon	Tune Sysmon, add PowerShell Script Block Logging, capture Event IDs 4625 and 4720 clearly
Ubuntu 22.04 — 192.168.56.103	auth.log, sudo, SSH, FIM	Add auditd and monitor sensitive files, user-management commands, and sudo execution
Kali VM — future	Not currently required	Use as dedicated attacker machine for controlled tests only

4. Improvement 1 — Clean GitHub Evidence

This is the fastest improvement and should be completed before sharing the project on LinkedIn or a resume.

- Keep only the strongest 10–15 screenshots in the README. Too many screenshots make the project look unorganized.
- Delete screenshots that show usernames, passwords, private emails, or unnecessary system information.
- Use clean lowercase filenames such as powershell-alert.png, fim-alert.png, mitre-dashboard.png.
- Keep all incident reports inside the Reports/ folder and link them from the README.
- Add a docs/ folder for extra explanations such as future improvements and monitoring architecture.

GitHub Item	Recommended Status
README.md	Keep as the main recruiter-facing homepage
Reports/	Store all six PDF incident reports
screenshots/	Store selected proof screenshots only
Rules/	Store custom rule examples and explain whether they are tested or planned
docs/	Store improvement plans and architecture notes

5. Improvement 2 — Add Raw Alert Evidence

A real SOC analyst does not only take dashboard screenshots. They also open the alert details and record the raw event fields. For each attack, add one screenshot or text export showing the important alert details.

Attack	Evidence to Add
Windows brute-force failed logins	Show Event ID 4625, target username, source IP, failed reason, and Wazuh rule
Suspicious PowerShell execution	Show process command line, parent process, rule 92057, and MITRE T1059.001
New local user creation	Show Event ID 4720, account created, creator account, and MITRE T1136.001

Linux SSH brute force	Show auth.log raw line, source IP, invalid username, rule 5710/2502
FIM /etc/hosts modified	Show file path, timestamp, checksum change, and rule 550
Linux sudo privilege escalation	Show user jazz, sudo command, rule 5402/5404, and MITRE T1548.003

Recommended README Addition

```
## Evidence Method
For each detection, I collected:
- Dashboard screenshot
- Alert details
- Rule ID and severity
- MITRE ATT&CK mapping
- Incident report PDF
- Analyst verdict and recommendations
```

6. Improvement 3 — Enable Wazuh Active Response

Right now the lab mainly detects attacks. A stronger SOC lab should also show response. Wazuh Active Response can automatically block a source IP after repeated brute-force attempts.

- Start with Linux SSH brute-force blocking because it is easier to test safely.
- Use a controlled attacker VM or known lab IP only.
- Capture before-and-after screenshots: alert fired, active response log, and blocked connection.
- Do not enable aggressive blocking on your real network. Keep it inside the host-only lab.

```
# Check active response log on Wazuh/Linux endpoint
sudo cat /var/ossec/logs/active-responses.log

# Evidence to capture
1. Brute-force alert in Wazuh
2. active-responses.log entry
3. Connection blocked from attacker IP
```

7. Improvement 4 — Strengthen Windows Monitoring

The Windows endpoint is already useful because it has Wazuh Agent and Sysmon. The improvement is to make Windows telemetry deeper and cleaner.

Improvement	Why It Helps	Evidence to Capture
Tune Sysmon config	Reduces noise and improves detection quality	Sysmon running screenshot and Event ID 1 process logs
Enable PowerShell Script Block Logging	Shows what PowerShell actually executed	Event Viewer PowerShell logs
Capture Event ID 4625 clearly	Stronger proof of Windows brute force	Failed logon event details
Capture Event ID 4720 clearly	Stronger proof of user creation/persistence	New user account event details
Add account-group monitoring	Shows privilege escalation or admin group changes	Administrator group changed alert

```
# PowerShell logging idea for future improvement
# Run as Administrator on Windows:
Set-ExecutionPolicy RemoteSigned
# Enable PowerShell Script Block Logging through Group Policy or registry
# Then confirm events in Event Viewer.
```

8. Improvement 5 — Add Linux auditd on Ubuntu

The Ubuntu endpoint currently uses Wazuh Agent, auth.log, sudo logs, FIM, and vulnerability inventory. Adding auditd will make Linux monitoring deeper by recording kernel-level activity such as command execution and sensitive file access.

auditd Rule Goal	Example	SOC Value
Monitor /etc/shadow access	-w /etc/shadow -p wa -k shadow_access	Helps detect credential-access behavior
Monitor /etc/passwd changes	-w /etc/passwd -p wa -k passwd_changes	Helps detect account manipulation
Monitor sudoers changes	-w /etc/sudoers -p wa -k sudoers_changes	Helps detect privilege escalation setup

Monitor useradd/usermod	-w /usr/sbin/useradd -p x -k user_mgmt	Helps detect persistence through new users
Monitor sudo execution	-w /usr/bin/sudo -p x -k sudo_exec	Adds deeper proof for sudo abuse

```
# Install auditd on Ubuntu endpoint
sudo apt update
sudo apt install auditd audispd-plugins -y
sudo systemctl enable auditd
sudo systemctl start auditd
sudo systemctl status auditd

# Check audit logs
sudo tail -50 /var/log/audit/audit.log
```

9. Improvement 6 — Test Custom Wazuh Rules Properly

Your current project mentions custom rule examples. To make this stronger, test at least one custom rule and show it firing with a custom rule ID such as 100001 or 100002.

Custom Rule	Test Activity	Proof Needed
PowerShell encoded command rule	Run a safe encoded command on Windows	Wazuh alert showing custom rule ID
Netcat/reverse shell pattern rule	Run a harmless test command that matches pattern	Wazuh alert showing custom rule ID
Linux sensitive file access rule	Access /etc/shadow in lab	Alert with rule ID and raw log

Important: only mark custom rules as completed after the dashboard shows your custom rule ID firing. Until then, keep them under Future Improvements or Planned Custom Rules.

10. Improvement 7 — Add a Dedicated Kali Attacker VM

A Kali VM will make the lab architecture easier to understand because one machine becomes the attacker, while Windows and Ubuntu remain monitored endpoints.

VM	Role	Example Tests
Kali Attacker	Controlled attacker machine	SSH brute force, SMB login attempts, Nmap scans
Windows Endpoint	Monitored victim	Failed logins, PowerShell activity, user creation
Ubuntu Endpoint	Monitored victim	SSH brute force, sudo activity, FIM changes
Wazuh Server	SOC platform	Alerting, MITRE mapping, reports

11. Improvement 8 — Add Vulnerability Remediation Workflow

Your vulnerability screenshots are strong because they show CVEs on Ubuntu and Windows. The next improvement is to show how an analyst would prioritize and remediate them.

- Take one high or critical CVE from the dashboard.
- Record package name, CVE ID, severity, and affected endpoint.
- Explain the remediation action, such as Windows Update or apt upgrade.
- Re-scan and compare the before/after vulnerability count.
- Document this as a vulnerability management mini-report.

Step	What to Document
Identify	CVE ID, affected package, affected endpoint, severity
Prioritize	Critical/high first, especially if exposed or exploitable
Remediate	Patch, update, remove software, or mitigate exposure
Verify	Run Wazuh scan again and compare results
Report	Summarize before/after risk reduction

12. Improvement 9 — Add Alert Notifications

A real SOC needs high-severity alerts to reach analysts quickly. Add email, Slack, or webhook notifications for important alerts such as brute force, PowerShell encoded command, or sudo abuse.

- Start with email alerting for level 10+ alerts.
- Create a screenshot showing notification received.
- Document which alerts should generate notifications.
- Avoid sending noisy low-level alerts.

13. Improvement 10 — Upgrade Reports and Documentation

The six incident reports are already a strong part of the project. Improve them by adding more analyst-style structure.

Report Section	Improvement
Executive Summary	Keep it short and clear for non-technical readers
Timeline	Add exact timestamps from Wazuh alerts
Detection Source	Include log source, rule ID, severity, agent, and event ID
Investigation Findings	Add raw evidence and analyst interpretation
MITRE Mapping	Explain why the activity maps to that technique
Response	Add what the analyst would do in production

14. Improvement 11 — Add New Alert Scenarios

After the current six detections are documented, the next upgrade is to add a few more alert scenarios. These alerts should be treated as controlled lab simulations only. Add them one at a time, capture the Wazuh alert, record the rule ID/severity, and map each alert to MITRE ATT&CK before adding it to the GitHub README.

Recommended New Alerts

Priority	New Alert	Endpoint	Main Log Source	MITRE Mapping

1	Windows administrator group changed	Windows 10	Windows Security Log	T1098 — Account Manipulation
2	Windows scheduled task created	Windows 10	Security Log / Sysmon	T1053.005 — Scheduled Task
3	Windows service installed or changed	Windows 10	System Log / Sysmon	T1543.003 — Windows Service
4	Linux new local user created	Ubuntu	auth.log / auditd	T1136.001 — Local Account
5	Linux SSH authorized_keys modified	Ubuntu	FIM / auditd	T1098.004 — SSH Authorized Keys
6	Linux cron persistence file modified	Ubuntu	FIM / auditd	T1053.003 — Cron
7	Sensitive file access: /etc/shadow	Ubuntu	auditd	T1003 — OS Credential Dumping

14.1 Windows Administrator Group Changed

Why this alert matters: This alert shows possible privilege escalation or account manipulation. Attackers often add a newly created account to the local Administrators group to gain higher privileges.

Safe lab simulation: Run only inside the Windows lab VM as Administrator. Create a test user first, then add it to the Administrators group.

```
net user labadmin P@ssw0rd123 /add
 net localgroup administrators labadmin /add
 net localgroup administrators labadmin /delete
 net user labadmin /delete
```

Evidence to capture: Capture the Wazuh Security Events alert showing Administrators group changed, the Windows agent name, rule ID, severity, and the account involved.

MITRE mapping: T1098 — Account Manipulation / Privilege Escalation

14.2 Windows Scheduled Task Created

Why this alert matters: Scheduled tasks are commonly abused for persistence. A SOC analyst should investigate unexpected task creation, especially if the task runs PowerShell, cmd.exe, or unknown scripts.

Safe lab simulation: Create a harmless scheduled task in the Windows lab VM, confirm Wazuh detects it, then delete the task.

```
schtasks /Create /SC ONCE /TN LabTestTask /TR "cmd.exe /c echo lab-test" /ST 23:59
 schtasks /Delete /TN LabTestTask /F
```

Evidence to capture: Capture the alert for schtasks.exe, scheduled task creation, or related Windows/Sysmon process creation. Record parent process, user, and command line if available.

MITRE mapping: T1053.005 — Scheduled Task / Job: Scheduled Task

14.3 Windows Service Installed or Changed

Why this alert matters: Attackers may create or modify Windows services for persistence or privilege escalation. This is a strong Windows endpoint detection to add after user creation and PowerShell detection.

Safe lab simulation: Create a harmless test service in the Windows lab VM, verify the event appears, then delete it.

```
sc create LabTestService binPath= "C:\Windows\System32\cmd.exe /c echo lab-service" start= demand
 sc delete LabTestService
```

Evidence to capture: Capture Windows service creation/change alert, service name, command line, user, endpoint, and any Sysmon process creation event.

MITRE mapping: T1543.003 — Create or Modify System Process: Windows Service

14.4 Linux New Local User Created

Why this alert matters: Linux user creation is a persistence technique. If an attacker gains shell access, they may create a new user and later add it to sudoers or the sudo group.

Safe lab simulation: Run this on the Ubuntu lab VM, then remove the user after capturing the alert.

```
sudo useradd labuser
 sudo passwd labuser
 sudo userdel -r labuser
```

Evidence to capture: Capture auth.log/Wazuh alert showing useradd or account creation. If auditd is added later, also capture the audit event.

MITRE mapping: T1136.001 — Create Account: Local Account

14.5 Linux SSH authorized_keys Modified

Why this alert matters: Attackers often add SSH keys for persistence. Monitoring authorized_keys is useful because it shows unauthorized SSH access setup.

Safe lab simulation: Create a harmless backup, append a test line, trigger FIM/auditd, then restore the file.

```
mkdir -p ~/.ssh
 cp ~/.ssh/authorized_keys ~/.ssh/authorized_keys.bak 2>/dev/null || true
 echo "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQDlabtest lab-key" >> ~/.ssh/authorized_keys
 mv ~/.ssh/authorized_keys.bak ~/.ssh/authorized_keys 2>/dev/null || rm ~/.ssh/authorized_keys
```

Evidence to capture: Capture FIM or auditd alert showing authorized_keys was modified. Record path, user, timestamp, and hash change if available.

MITRE mapping: T1098.004 — Account Manipulation: SSH Authorized Keys

14.6 Linux Cron Persistence File Modified

Why this alert matters: Cron jobs are common Linux persistence mechanisms. Wazuh FIM or auditd can detect changes to cron directories and cron files.

Safe lab simulation: Create a harmless cron file and then remove it after alert capture.

```
echo "* * * * * root echo lab-cron-test > /tmp/lab-cron.txt" | sudo tee /etc/cron.d/lab-test
 sudo rm /etc/cron.d/lab-test
```

Evidence to capture: Capture FIM/auditd alert showing /etc/cron.d/lab-test was created or modified. Record file path and rule details.

MITRE mapping: T1053.003 — Scheduled Task/Job: Cron

14.7 Sensitive File Access: /etc/shadow

Why this alert matters: Access to /etc/shadow can indicate credential access. This alert becomes stronger after auditd is added because auditd can show file access activity more clearly.

Safe lab simulation: Run a controlled read attempt on Ubuntu and capture auth/audit evidence. Do not expose or upload the contents of /etc/shadow to GitHub.

```
cat /etc/shadow
 sudo cat /etc/shadow > /dev/null
```

Evidence to capture: Capture only the alert/log evidence, not the file contents. In GitHub, blur or avoid any screenshot showing password hashes.

MITRE mapping: T1003 — OS Credential Dumping

14.8 README Table Addition for New Alerts

After these alerts are tested, add a new section to the GitHub README. Keep the status as Future or Planned until you have screenshots and rule details.

```
## Additional Alert Scenarios Planned

 | Alert Scenario | Endpoint | Log Source | MITRE ID | Status |
|---|---|---|---|---|
| Administrator group changed | Windows | Windows Security Log | T1098 | Planned |
| Scheduled task created | Windows | Security Log / Sysmon | T1053.005 | Planned |
| Windows service installed | Windows | System Log / Sysmon | T1543.003 | Planned |
```

Linux local user created	Ubuntu	auth.log / auditd	T1136.001	Planned
SSH authorized_keys modified	Ubuntu	FIM / auditd	T1098.004	Planned
Cron persistence file modified	Ubuntu	FIM / auditd	T1053.003	Planned
/etc/shadow access detected	Ubuntu	auditd	T1003	Planned

14.9 New Alert Checklist

- ☐ Test each new alert one at a time inside the isolated VirtualBox host-only lab.
- ☐ Capture Wazuh dashboard screenshot and raw alert details for each alert.
- ☐ Record rule ID, rule description, severity level, agent name, username, and timestamp.
- ☐ Map every new alert to MITRE ATT&CK before adding it to the README table.
- ☐ Do not upload screenshots containing passwords, hashes, or private information.
- ☐ Mark the new alerts as Planned until screenshots and evidence are available.

15. Final Improvement Checklist

- ☐ Delete any screenshot that shows passwords or private information.
- ☐ Keep README screenshots focused and professional.
- ☐ Add a docs/ folder with this improvement plan.
- ☐ Add raw alert detail screenshots for each of the six attacks.
- ☐ Enable and document Wazuh Active Response in the lab.
- ☐ Add auditd to Ubuntu and capture audit evidence.
- ☐ Enable stronger PowerShell logging on Windows.
- ☐ Test at least one custom Wazuh rule and show it firing.
- ☐ Add a Kali attacker VM as a dedicated attack source.
- ☐ Create a vulnerability remediation mini-report.
- ☐ Add alert notification proof for high-severity alerts.
- ☐ Pin the GitHub repo and add it to the GitHub profile README.

16. Strong Resume Version After Improvements

After completing the above improvements, use this stronger resume bullet:

> Built and improved a mini enterprise SOC lab using Wazuh SIEM/XDR with Windows Sysmon and Ubuntu Linux endpoints. Configured log collection, MITRE ATT&CK mapped detections, File Integrity Monitoring, vulnerability detection, Active Response planning, Linux auditd improvements, and formal incident reports for six simulated attack scenarios.

17. Final Recommendation

Do not try to add everything at once. First fix the GitHub evidence, then add raw alert details, then implement Active Response, then add auditd and custom rules. This order will make the project cleaner and easier to explain during interviews.